LEGEND

Geo Embeddings 101

Models, image chips, and semantic search

Matt Hanson

FOSS4G North America 2026 · Sacramento

Nov 4, 2026

You have likely heard all of these

- embedding
- vector embedding
- image embedding
- Earth observation embedding
- geo embedding
- geospatial embedding
- embedding field
- receptive field
- latent space
- foundation model
- geospatial foundation model
- encoder
- fine-tuning
- zero-shot
- semantic search
- pixel-based model
- matryoshka embeddings

Now try defining any of them

01

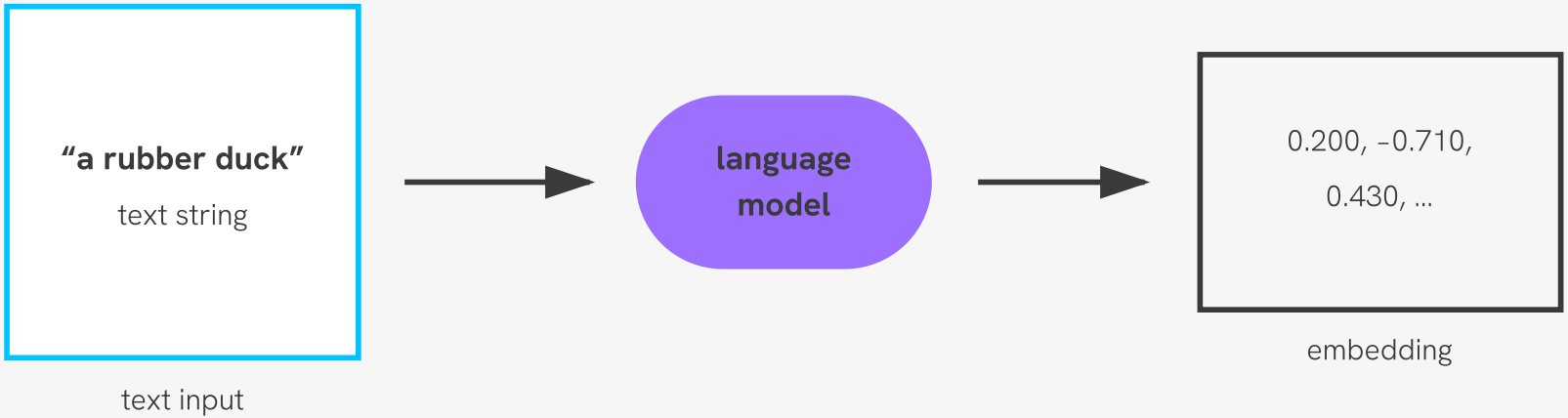
What is an embedding?

Text, images, and other data can become vectors

An embedding is a

summary

Language models



A text-embedding model produces one vector for the whole string.

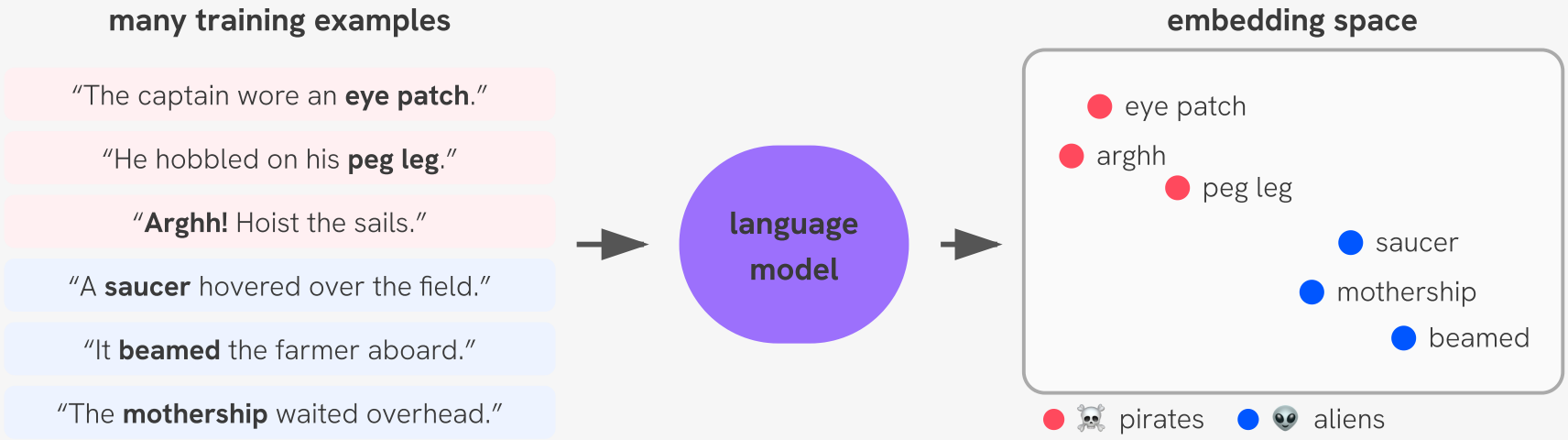
Every token reads the whole sentence

A flying **saucer** hovered over the field. → `0.31, −0.08, …`

He set the cup back on its **saucer**. → `−0.12, 0.44, …`

Same word, two different vectors. Every other token in the sentence feeds into it, not a fixed window of neighbours, and not the word on its own. Combine all of them and you get one vector for the whole sentence.

What did the model learn?



- Each highlighted word gets its own vector, shaped by the sentence around it.
- None of these words means *pirate* or *alien* on its own. “Hoist the sails” is about sailors, and pirates are sailors.
- This is what **semantic** similarity means.

Now replace the text with an image

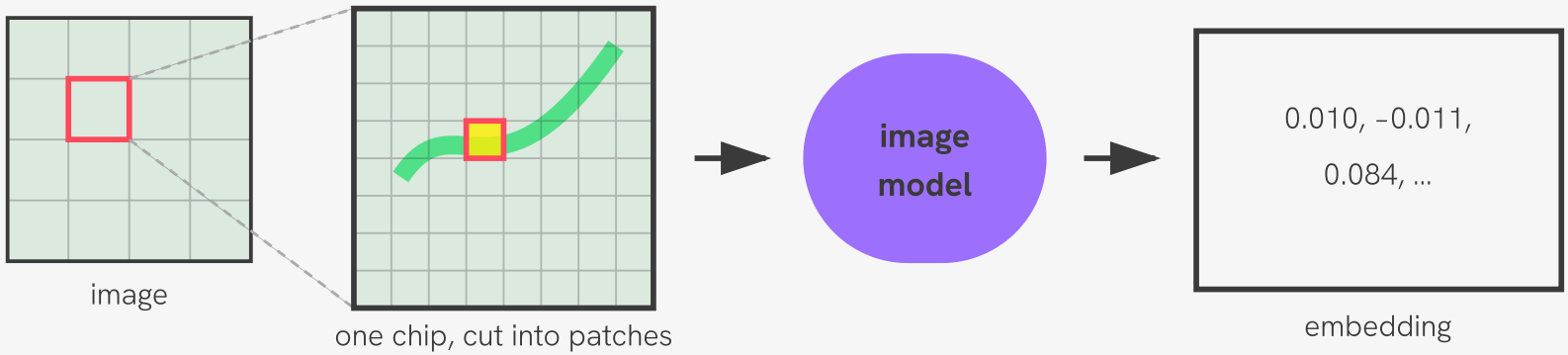
NAIP · Alabama, 2023 · 1,024 × 1,024 px

Same idea, different model

- The input is an **image chip**: a fixed-size crop of overhead imagery, not a sentence
- The model is a **geospatial model**, trained on overhead imagery: Sentinel-2, Landsat, NAIP
- Nothing in it was trained on text
- The output is the same shape: one vector

Both models turn an input into numbers that preserve useful meaning. They do not produce the same numbers, and they do not share a vector space.

Image → vector



The model cuts the chip into a grid of **patches**, then reduces them to one vector for the chip.

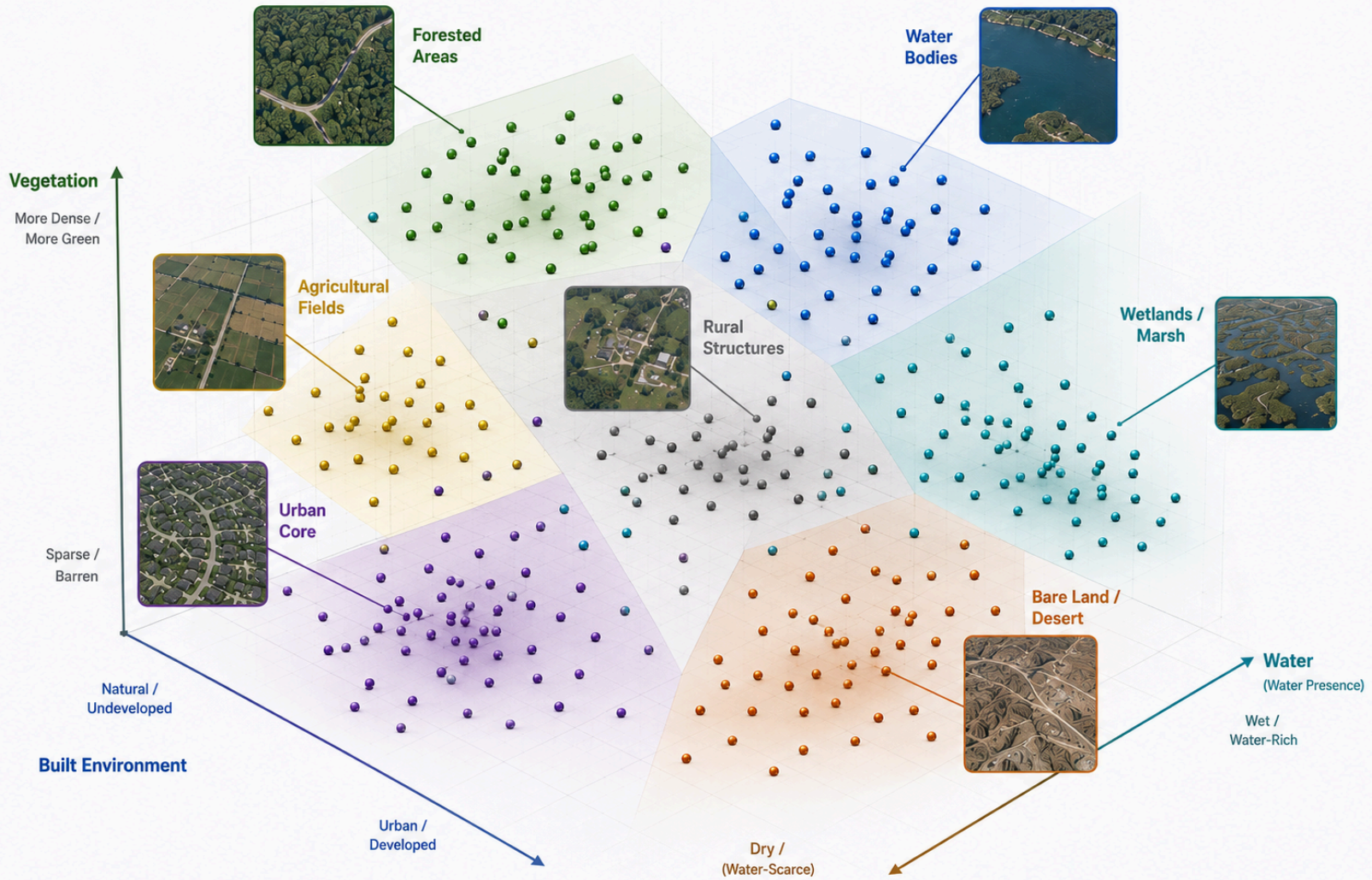
The same structure in text and imagery

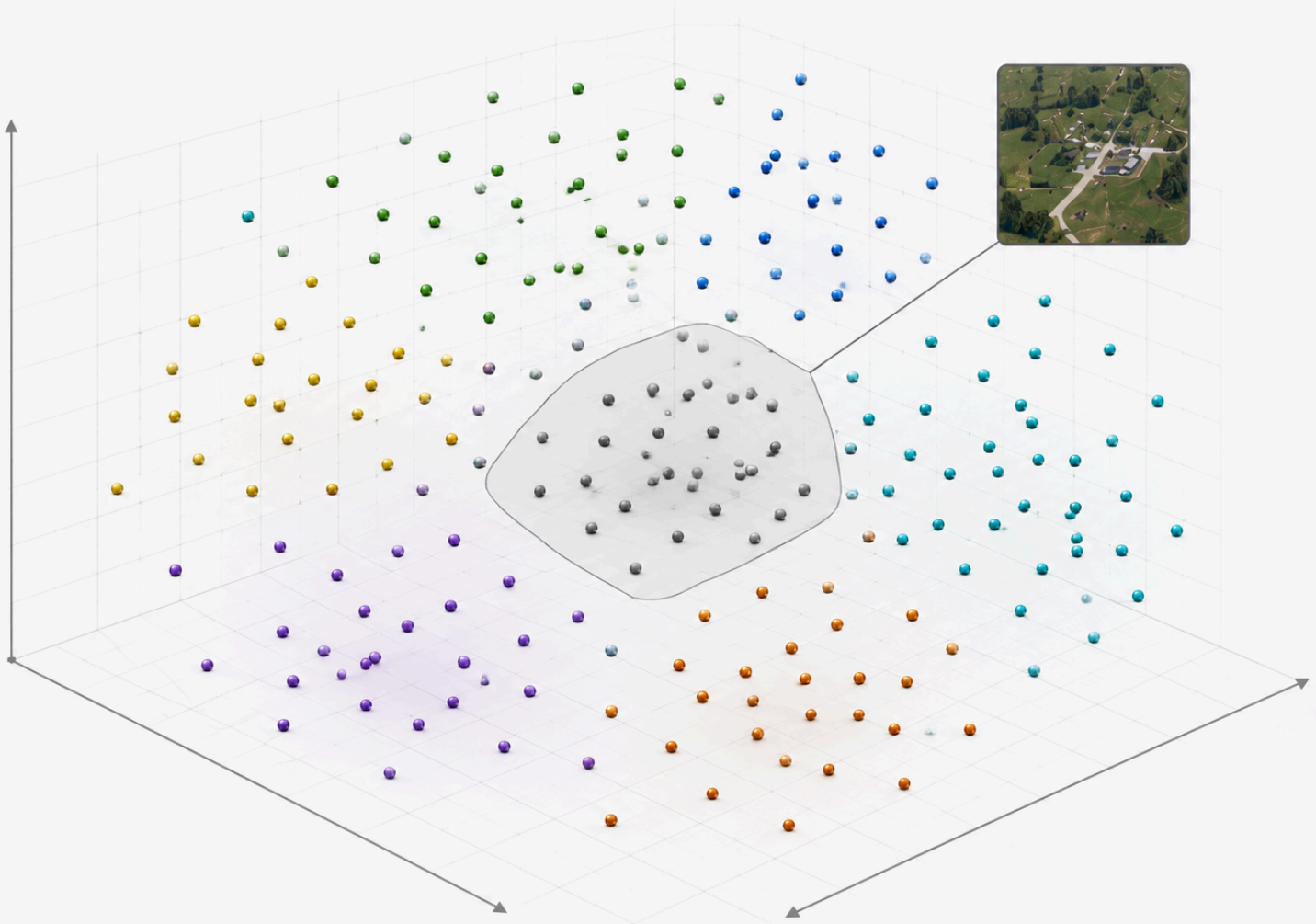
Text	Imagery
a sentence	a **chip**
a token	a **patch**
one vector per token	one vector per patch
one vector for the sentence	one vector for the chip

Most models hand you the chip vector. Some hand you the patch grid instead, and some skip chips entirely and give one vector per pixel. (“Patch” is an imagery word. Text only ever says token.)

What kind of embedding?

Term	What it means
Embedding · vector embedding	The same thing. Every embedding is a vector of numbers, and "embedding" is the shorthand
Image embedding	Input: pixels
Earth observation embedding	Input: pixels, spectral bands, resolution, and sometimes time
Geospatial embedding	Input: Earth observation data plus explicit location or place context
Embedding field	Measurements anchored across space and time
Latent space · embedding space	Not a vector. The space the vectors live in


Forested Areas
Water Bodies
Vegetation
More Dense / More Green
Agricultural Fields
Rural Structures
Wetlands / Marsh
Urban Core
Sparse / Barren
Bare Land / Desert
Water
(Water Presence)
Wet / Water-Rich
Natural / Undeveloped
Built Environment
Urban / Developed
Dry / (Water-Scarce)



What is an embedding?

They are summaries

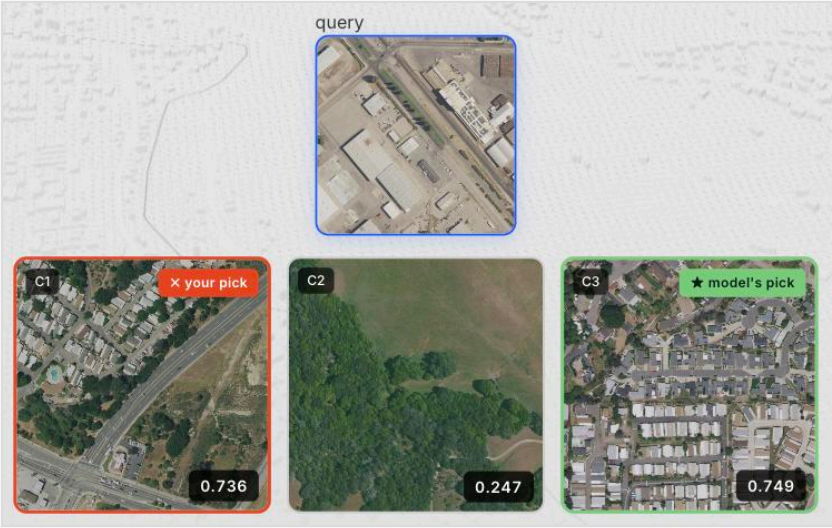
They are imperfect summaries

They are imperfect but detailed summaries

They are imperfect but detailed mathematical summaries

They are not classifications

Guess what Clay thinks



One query chip, six candidates. Pick the one you think sits closest in Clay's embedding space, then see the cosine similarity for every option.

Here the model and the player disagreed: 0.749 against 0.736. Both suburbs. The forest scored 0.247.

chipmatch.apps.lgnd.hosted.unionai.cloud

Sam Barrett, LGND

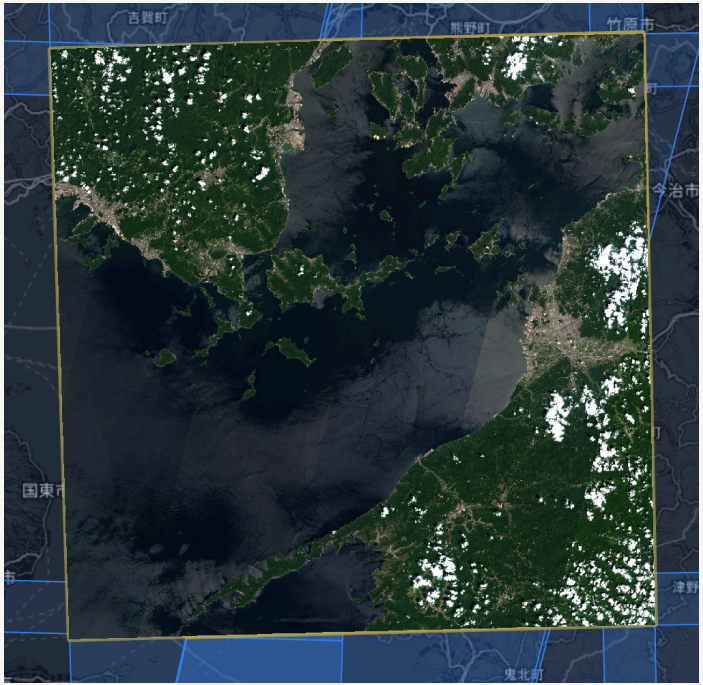
02

Chip size

Footprints, overlap

Why not embed a whole Sentinel-2 image?

- One Sentinel-2 tile is about **110 km across**, roughly 12,000 km^2
- A single vector for all of it would average a city, a forest, the sea and clouds into one summary
- At 256 px and 10 m, that same tile is about **1,800 chips**



Sentinel-2 · 10 m · Seto Inland Sea · 23 Aug 2026

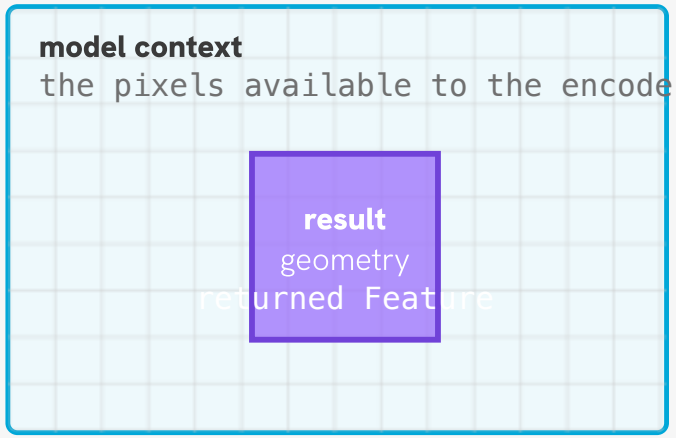
The chip is the

unit of search

Embedding footprint and model context

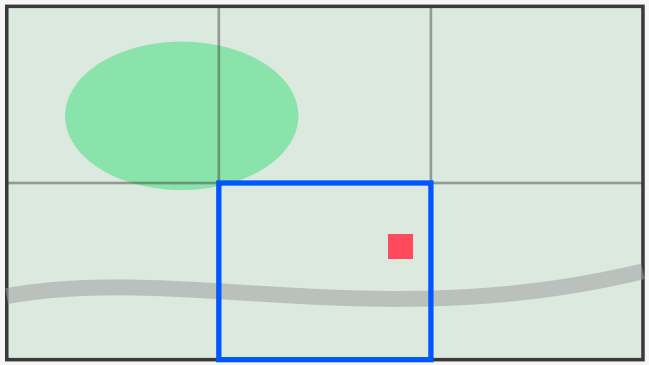
Suppose the search result is one **10 m pixel**.
The model may have produced that vector from a **2.56 km image chip** around it.

- `geometry` - footprint the embedding represents
- `context_geometry` - the area the model looked at

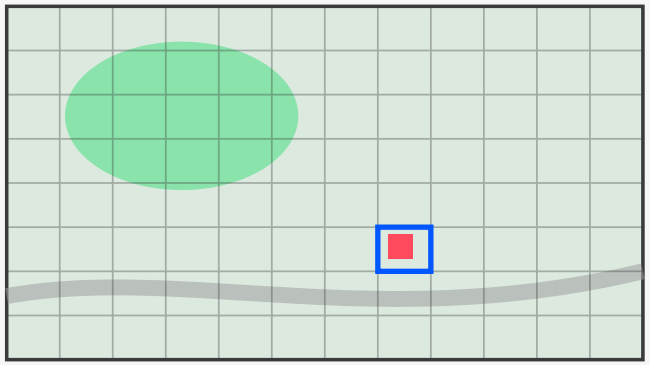


For chip embeddings, these may be the same. For pixel or patch embeddings, they may be very different.

The smallest thing you can find



large chips
lost in the average



small chips
most of its own vector

Same place, same imagery, two chip sizes. The red square is the thing you are looking for; the blue outline is the chip whose vector has to carry it.

So pick the chip size closest to the size of the thing you care about, and if you do not know it yet, pick smaller.

Chip size changes the question

NAIP

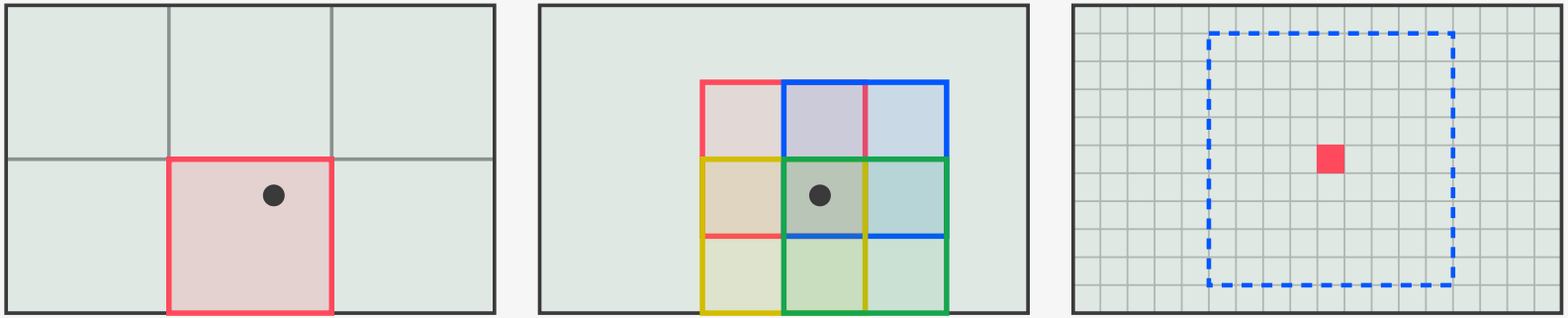
~1 m resolution · ~256 m across

Sentinel-2

10 m resolution · 2.56 km across

Both are 256 × 256 pixels.

Three ways to cover the ground



no overlap
1 vector for this place

50% overlap
4 vectors for this place

per pixel
1 vector, read from wider

The dot is one place; the outlines are the chips that have to describe it.

Overlap is an inference choice, not a property of the model. Record chip size **and** stride. Per-pixel goes furthest: what a vector **describes** shrinks to one pixel while what it **reads** stays wide.

Adding time

	One image	A time series
The model reads	one date	many dates, in order
One vector describes	that day	the whole period
Examples	Clay, TerraMind, DINOv3 SAT	TESSERA, AlphaEarth, OlmoEarth, Prithvi
Also record	the date	the window, and which dates went in

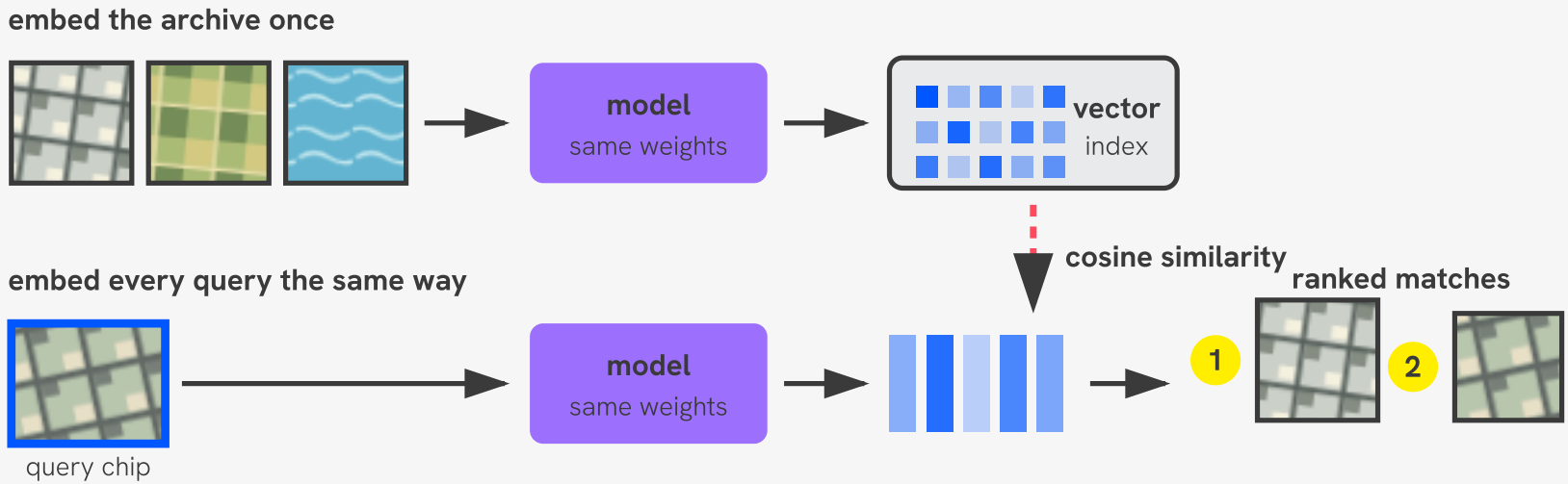
CONTEXT HAS A TIME EXTENT TOO.

03

Similarity and search

Queries, nearest neighbours, and cosine similarity.

Similarity search



The query must be embedded with the **same model** as everything already stored, then compared against every stored vector.

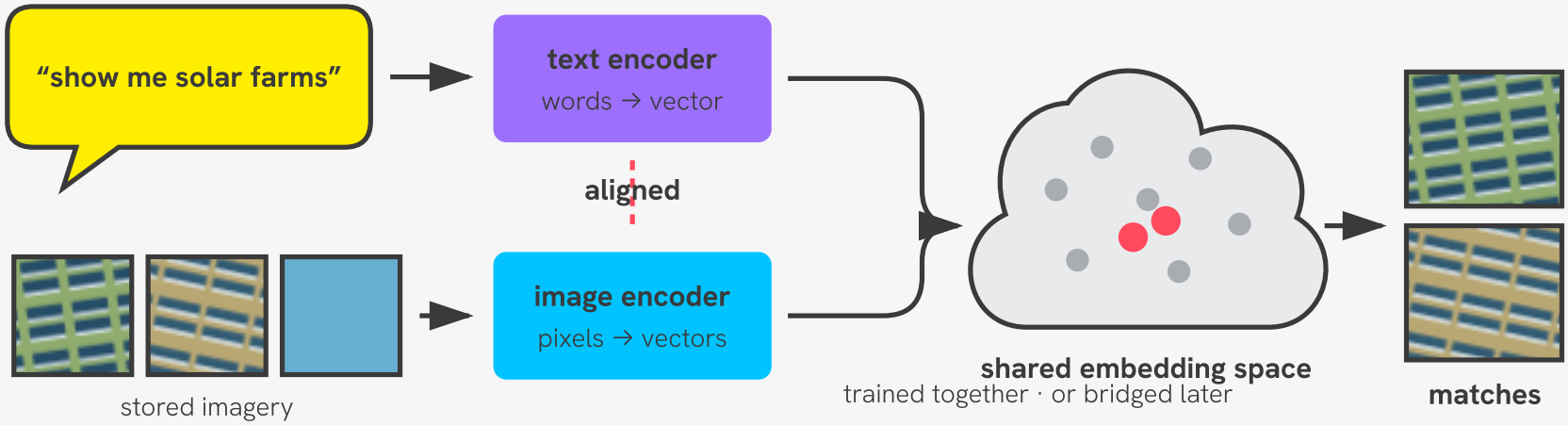
Similar imagery

query

nearest matches in the embedding space

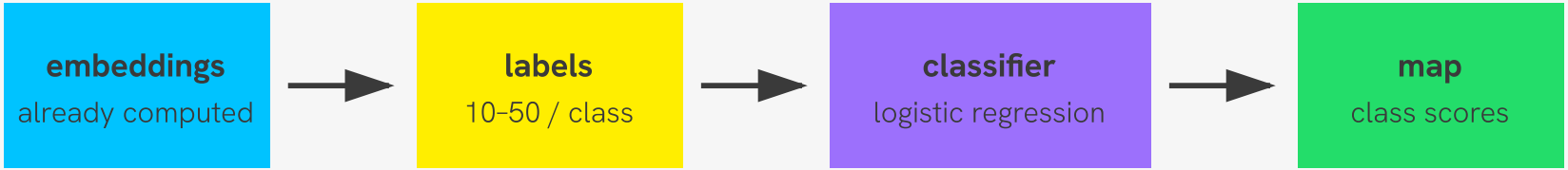
No labels. No classifier. One example, ranked by similarity.

What if my query is text?



Text search works only when the text encoder and image encoder share a vector space.

From embeddings to a classifier



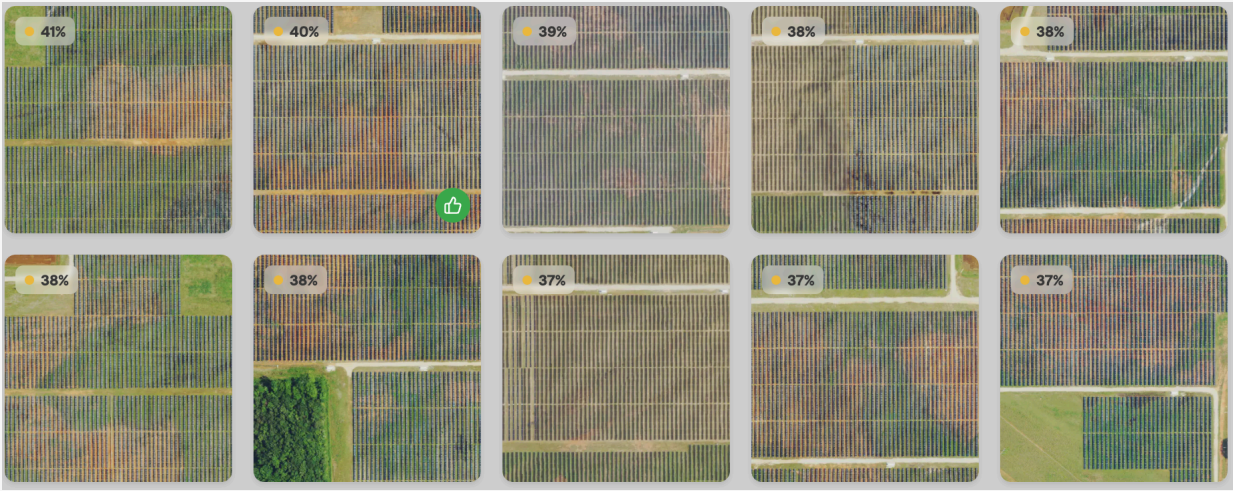
Most of the model training has already happened.

How many labels do you need?

Approach	Labels needed	What gets trained
Zero-shot	none	Nothing. You compare vectors directly. Similarity search is this
Few-shot	10–50 per class	A small classifier on top of frozen embeddings
Fine-tuning	many more	The encoder itself

Fine-tuning changes the encoder, and a changed encoder means a changed vector space. Every vector you already stored becomes invalid.

Ten labels, every solar farm



Five positive and five negative chips. The whole training set, and fewer per class than the table above suggests. The ranking is a **classifier score, not a distance and not a percentage**. Raw scores run −0.34 to 0.44: read the order, not the number.

04

Geospatial Foundation Models

Input, training data, chip contract, output, time, and openness.

What is a foundation model?

Term	What it means
Encoder	The part that turns an input into learned features: chip, patch, or pixel vectors
Foundation model	Trained once, on a large amount of data, for no particular task, then reused for many tasks
EO / geospatial foundation model	A foundation model trained on Earth observation imagery, sometimes with location as an input

The expensive training already happened. You are using the result.
Text search needs a text encoder aligned to the image features; imagery training alone is not enough.

Inputs and training

Model	Input	Training data	Context: space · time
Clay 1.5	S2, Landsat, S1, NAIP, LINZ, MODIS	70M chips; corpus not packaged	256 px chip, 2.56 km at 10 m · one date
AlphaEarth	optical, radar, LiDAR/elevation, climate, land cover, text	many public sources; corpus not released	around each pixel, size not published · a year
TESSERA v2	S1 + S2	annual radar + optical sequences	one pixel · a year
Prithvi 2.0	six HLS bands	4.2M HLS v2 samples	224 px, 6.7 km at 30 m · four dates
TerraMind Base	S1, S2, DEM, RGB	9M published TerraMesh samples	224 px, 2.24 km at 10 m · one date
OlmoEarth 1.3 Base	S2, S1, Landsat	published pretraining data	up to 128 px · one date or up to a year
DINOv3 SAT	RGB	undisclosed SAT-493M corpus	256 px RGB · one image

What might cluster in each space?

Model	One vector	Nearby vectors might share…
Clay 1.5	Chip, one date	things you can see in one image: solar farms, quarries, centre pivots, ports
AlphaEarth	Pixel, one year	what the land is all year: cropland, wetland, forest type, built-up
TESSERA v2	Pixel, one year	how a pixel changes through the year: crop type, irrigation, seasonal flooding
Prithvi 2.0	Chip, four dates	what changed across the dates: burn scars, floods, crop growth
TerraMind Base	Chip, one date	land cover and terrain: forest, cropland, water, mountains
OlmoEarth 1.3 Base	Chip, one date or a year	things in one image, or crops and land use across a year
DINOv3 SAT	RGB chip, one image	shapes: buildings, roads, field boundaries

Embedding Dimensions

	Dimensions	A billion vectors
AlphaEarth	64	256 GB
TESSERA · OlmoEarth	128	512 GB
Earth Index	384	1.5 TB
Clay 1.5	1,024	4 TB

FLOAT32. THE PUBLISHED INT8 FILES ARE A QUARTER OF THAT.

Can I keep just the first 64?

- **Matryoshka-trained:** yes. TESSERA v2, OlmoEarth 1.3
- The first 16, 32, 64 are each a usable embedding, like nested dolls
- **Not trained for it:** no guarantee. Check the model card
- Otherwise: **PCA**, fitted once, applied to every vector
- Re-normalise after cutting. Never compare 64 with 128

Outputs and licenses

Model	Output	Time	What is open?
Clay 1.5	1,024-d CLS and patch tokens	single observation	Apache-2.0 weights + code
AlphaEarth	64-d per 10 m pixel	annual	CC BY outputs; model closed
TESSERA v2	128-d per 10 m pixel	annual sequence	CC0 weights; MIT code
Prithvi 2.0	768–1,280-d tokens, by model size	four dates	Apache-2.0
TerraMind Base	196 × 768 patch tokens per modality	supplied observation	Apache-2.0; corpus CC BY-SA
OlmoEarth 1.3 Base	768-d tokens; 128-d per 10 m pixel product	single image, series, or annual	weights, code, data, embeddings; custom licence
DINOv3 SAT	class and patch tokens	single image	gated, custom terms

“Open” is several questions

- Are the **weights** available?
- Is the inference **code** available?
- Can you inspect the **training data**?
- Do the **licences** permit your use?
- Can you run **inference** yourself?

AlphaEarth has open outputs from a closed model. TerraMind publishes weights, code, and its training corpus. OlmoEarth publishes all three as well, but its licence forbids military and extractive use, so it is not open source. Clean examples in this survey include **CROMA, SeCo, and UniverSat**. Among the models above, **TerraMind** comes closest: open weights, code, inference, and the actual training corpus.

05

How do I use them?

Start with precomputed vectors, then decide whether you need to run a model.

A model or a dataset?

	A model	Pre-computed embeddings
You get	weights and code	files of vectors
Imagery, dates, chip size	your choice, within what the model accepts	already chosen
You need	the imagery and a GPU	something that reads Parquet or Zarr
New places or dates	yes	only what they ran

ONE MODEL, MANY DATASETS. SOME DATASETS HAVE NO PUBLIC MODEL.

Pre-computed embeddings

Dataset	Made with	One vector per	Model read	Dims	Coverage	Licence
LGND · Sentinel-2	Clay 1.5	1,280 m chip, per month	the chip · each scene, aggregated monthly	256	global, 2017 to Apr 2026	CC BY 4.0
AlphaEarth	AlphaEarth (closed)	10 m pixel, per year	around the pixel · the year	64	global, 2017–2025	CC BY 4.0
TESSERA	TESSERA v1.1	10 m pixel, per year	the pixel only · the year, S1 + S2	128	global, 2017–2025	CC0
OlmoEarth	OlmoEarth 1.3	10 m pixel, per year	around the pixel · the year, 30-day mosaics	128	global, 2021–2025	custom
Earth Index	SoftCon	~320 m chip	the chip · a 2024 mosaic	384	global, 2024	CC BY 4.0

ALL ON SOURCE COOPERATIVE.

Where should I start?

If you want to…	Start with…	Why
explore search or classification	**Source Cooperative**	precomputed; no GPU
compare models on the same chips	**Major TOM 249k suite**	six models, one grid
use annual per-pixel vectors	**AlphaEarth, TESSERA or OlmoEarth**	the published output is already a map
embed your own multispectral imagery	**Clay, TerraMind, Prithvi, OlmoEarth or DOFA**	control imagery, chip size and pooling
embed RGB imagery	**Clay or a satellite vision model**	simpler input contract

Precomputed embeddings are the fastest start. Run inference when you need control or new coverage.

Similarity from an existing Clay chip

```sql
WITH chips AS MATERIALIZED (
  SELECT chips_id, embedding
  FROM read_parquet('s3://us-west-2.opendata.source.coop/clay/lgnd-embeddings/' ||
    'monthly-aggregated/model_version=v1.5/collection=sentinel-2-l2a/' ||
    'chip_size=1280m/dims=256/geohash=9y/year=2025/month=01/*.parquet')
),
seed AS (SELECT * FROM chips ORDER BY chips_id LIMIT 1)
SELECT c.chips_id,
       list_cosine_similarity(c.embedding, s.embedding) AS score
FROM chips c CROSS JOIN seed s
WHERE c.chips_id <> s.chips_id
ORDER BY score DESC
LIMIT 12;
```

Here the query is one existing Clay vector. For your own chip, run **Clay 1.5 with the same Sentinel-2 configuration**, then supply its 256 values instead. An unrelated image or text vector will not work.

Every model summarises differently

Choose one that captures

what you're looking for

WHICH SENSORS? HOW BIG A CHIP? ONE DATE OR A YEAR?

lgnd.ai

Thank you

What will you use embeddings for?

SLIDES

geoskeptic.dev/presentations/2026/open-geo-embeddings

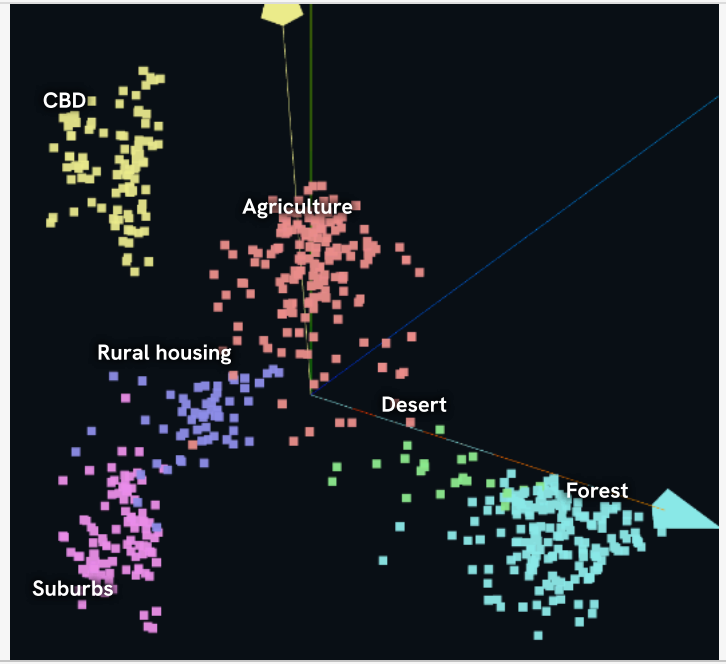
matt@lgnd.ai

What are embeddings useful for?

YOU NEED TO…	START WITH…
Find places that look like this one	EMBEDDINGS
Find sites where something is happening	EMBEDDINGS
Detect semantic change	EMBEDDINGS
Count instances of something	EMBEDDINGS + SEGMENTATION
Search a large archive quickly	EMBEDDINGS
Measure reflectance, temperature, or biomass	ARD
Quantify change: 20% biomass loss	ARD
Produce an authoritative or regulatory measurement	ARD
Maintain a calibrated long-term record	ARD

ARD = analysis-ready data. Embeddings find patterns; ARD preserves measurements.

The classifier creates these axes



Clay gives each chip 1,024 numbers. None of those dimensions means “forest,” “CBD,” or “agriculture.” We labelled 642 chips, then fitted a linear classifier for each class. Each classifier found a **direction through the existing embedding space**; three of those fitted directions become the axes in this picture.

The classes separating along those directions is the point: the useful structure was already in Clay’s embeddings. The classifier only learned where to draw the boundaries.

Embedding Geometry Toy · Sam Barrett. The concept directions are linear SVMs fitted on human-labelled chips, not something Clay ships.

The STAC Embeddings extension

v1.0.0 · 28 Oct 2026 · Proposal

Choice	Field
Which model	`emb:encoder_model` link
Pixel or chip	`emb:type`
Vector size	`emb:dimensions`
Chip size, stride	`emb:chip_size` · `emb:stride`
Grid	`emb:grid_id`
Quantized?	`emb:quantization_method`
Time window	`emb:temporal_period`
The imagery it came from	`derived_from` links

GITHUB.COM/STAC-EXTENSIONS/EMBEDDINGS

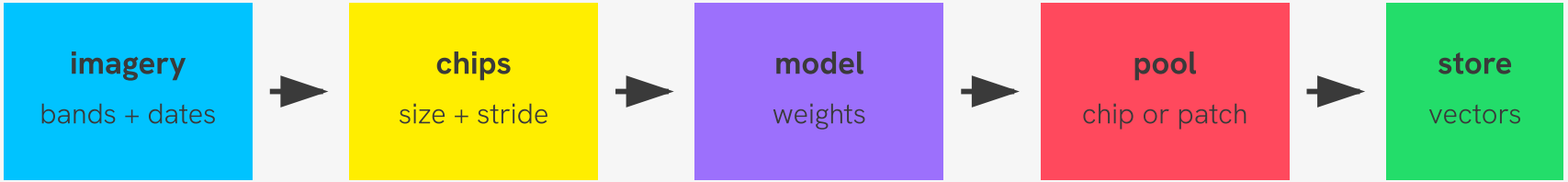
Embeddings on Source Cooperative

Dataset	What you get	Format / open with
Clay 1.5 · Sentinel-2	global monthly chip vectors; 1,280 m / 256-d or 2,560 m / 1,024-d	Hive-partitioned **GeoParquet** · DuckDB, GeoPandas
AlphaEarth · annual	64-d per 10 m pixel, 2017–2025, tiled by year and UTM zone	quantized **COG + correcting VRT** · GDAL, Rasterio, QGIS
AlphaEarth · mosaic	the same annual 64-d pixels as one global `(time, band, y, x)` array	sharded **Zarr v3** · Zarr, Xarray
OlmoEarth 1.3 · annual	128-d per 10 m pixel, 2021–2025; the first 64 work alone	**Zarr v3**, one group per UTM zone · Zarr, Xarray

`https://data.source.coop/<organization>/<repository>/<path>`

For Clay, filter Hive partitions and bounding boxes. For AEF COGs, use the supplied spatial index and open the **VRT**, not the TIFF directly. AEF and OlmoEarth store quantized `int8`. Dequantize first.

Running your own inference



Pooling is how a model reduces its internal patch vectors to one vector for the chip, usually a mean, or a **CLS** token added for the purpose. Most tools hand you the chip vector and you never see a patch.
Every choice becomes part of the resulting dataset.